

Sudoku-Tool

Lena Marie Ludwig, Jari Hartmann, Lisa Rappold

Technische Hochschule Ulm

Juli 2026

1 Einleitung

Dieses Projekt hat das Ziel, ein Sudoku-Tool zu entwickeln, mit dem man interaktiv Sudoku spielen kann oder eine Funktion zur direkten Lösung des Rätsels bietet.

Das Programm kann ein Sudoku aus einer Textdatei einlesen und dieses auf Gültigkeit prüfen. Alternativ kann man ein Sudoku mit beliebiger Schwierigkeitsstufe generieren lassen. Anschließend kann der Benutzer entweder interaktiv das Sudoku lösen, oder das Sudoku mit einem von zwei Lösungsalgorithmen lösen lassen. Abschließend kann die Lösung des Sudokus gespeichert werden. In diesem Bericht stellen wir die Umsetzung und Funktionalität der einzelnen Funktionen vor.

2 main() (Jari)

To do: Jari

3 read() (Lisa)

Eine `read()` Methode zum Einlesen von Sudoku-Rätseln ist essenziell für die Funktionalität des Solvers. Hierfür gibt der User sein Sudoku in die Textdatei `inputSudoku.txt` ein. Anschließend wird versucht, die Textdatei im Lesemodus zu öffnen. Ist das Öffnen erfolgreich, liest die Methode mithilfe einer `while`-Schleife das Sudoku ein. Hierbei werden kleine Fehler bei der Nutzereingabe ignoriert. Leerzeichen, senkrechte Striche, Bindestriche und leere Zeilen werden übersprungen. Bereits bei dem Einlesen wird eine grundlegende Validierung vorgenommen. Die Methode wirft eine Fehlermeldung, wenn in einer Zeile, die nicht leer ist, mehr oder weniger als 9 Zahlen vorkommen. Außerdem werden nur Eingaben von 0 bis 9 akzeptiert. Die Zahl 0 steht hierbei für ein leeres Feld. Auch bei zu vielen oder zu wenigen Zeilen wird eine Fehlermeldung ausgegeben. Handelt es sich bei dem eingegebenen Zeichen um eine Zahl zwischen 0 und 9, wird durch die Subtraktion des ASCII Wertes von 0 der korrekte Wert in der Matrix gespeichert. Zuletzt ruft die Funktion `printMatrix()`, um die eingelesene Matrix in der Konsole auszugeben.

4 create() (Lisa)

Stattdessen kann man sich auch mithilfe von `create()` ein zufälliges, eindeutiges Sudoku generieren lassen. Hierfür werden zwei zufällige Zahlen in ein leeres Sudoku eingetragen. Nachdem mithilfe der Funktion *Donald-Knuth* auf Gültigkeit geprüft wurde und ein Sudoku erstellt wurde, werden dann je nach Schwierigkeit mehr oder weniger Zahlen entfernt. Hierbei wird nach jeder entfernten Zahl geprüft, ob das Sudoku noch eindeutig lösbar ist, sonst wird eine andere Zahl entfernt.

5 validate() (Lisa)

Die Funktion `validate()` überprüft, ob das Sudoku gültig ist. Hierfür werden drei 9x9 Hilfs-Matrizen mit Nullen für Zeilen, Spalten und Blöcke erstellt.

Wird beispielsweise die erste Zeile des Sudokus überprüft, geht die Funktion jedes Element der ersten Zeile durch. Ist eine Zahl vorhanden, wird in der entsprechenden Zeile der Matrix *zeile* der Wert in Index *zahl-1*

auf 1 gesetzt. Ist dieses Element in der Matrix schon eine 1, kommt diese Zahl doppelt vor und es wird eine Fehlermeldung ausgegeben.

Indem die Indizes der verschachtelten for-Schleifen getauscht werden, können die Spalten des Sudokus überprüft werden. Es wird mit der selben Logik vorgegangen. Hierbei repräsentiert jede Spalte in der Matrix *spalte* eine Spalte in dem Sudoku.

Das Validieren der Blöcke ist etwas komplexer. Hierfür werden zwei Hilfsarrays BLOCKZEILE und BLOCKSPALTE erstellt, die für jeden der neun Blöcke die Koordinaten der linken oberen Ecke speichern. Die Zeile wird ermittelt, indem der entsprechende Wert aus BLOCKZEILE mit $j/3$ addiert wird. Da Nachkommastellen von Integern abgeschnitten werden, werden die entsprechenden Werte in BLOCKZEILE mit 0, 1 und 2 addiert, was die korrekte Zeile ergibt.

Die Spalte wird ermittelt, indem der entsprechende Wert aus BLOCKSPALTE mit $j\%3$ addiert wird. Hierdurch wird BLOCKZEILE mit der Sequenz 0, 1, 2 addiert, was die korrekte Spalte ergibt.

Hierbei wird jeder Block in der Matrix *block* in einer Zeile gespeichert. Auch hier repräsentiert jede Spalte eine Zahl.

6 solve() (Lena)

To do: Lena

7 solve() mithilfe des Algorithm X (Lisa)

Alternativ kann ein Sudoku auch als Exact-Cover-Problem betrachtet und mit dem Algorithm X von Donald Knuth gelöst werden. Hierfür wird mithilfe von `calloc()` und `malloc()` eine Matrix der Größe 729x324 erstellt. In den Zeilen dieser Matrix werden alle möglichen Züge repräsentiert. In den Spalten werden für jeden Zug die vier grundlegenden Sudoku Regeln gespeichert: Jedes Feld muss belegt sein, in jede Zeile und Spalte kommt jede Zahl einmal vor, und pro 3x3 Block kommt jede Zahl genau einmal vor. Am Ende sind also in jeder Zeile vier Einsen (abgesehen von einem Sonderfall).

Außerdem wird ein Array *coveredColumns* eingeführt, welches die bereits erfüllten Regeln speichert. Ist in jedem Index dieses Arrays genau eine Eins, ist das Sudoku valide gelöst. Es gibt auch ein Array *solution*, welches die verwendeten Zeilen von der großen Matrix speichert.

Eine Hilfsfunktion *solveMatrix()* nutzt dann Rekursion und Backtracking, um eine Lösung zu finden. Hierbei wird immer die nächste Null in *coveredColumns* gesucht, dann wird die große Matrix nach einem möglichen Spielzug durchsucht, die diese Null abdeckt und gegen keine andere Regel verstößt. Wird eine Zeile gefunden, wird diese in *solution* und *coveredColumn* gespeichert. Falls *coveredColumns* noch nicht ausschließlich mit Einsen gefüllt ist, aber trotzdem keine passende Regel vorhanden ist, wird der letzte Spielzug rückgängig gemacht und die Rekursion läuft weiter.

8 output() (Lisa)

Diese Funktion hat das Ziel, das Sudoku formatiert in einer Textdatei zu speichern.

Hierfür öffnet sie zunächst die Zielfeld *outputSudoku.txt* im Schreibmodus. Schlägt dies fehl, wird eine Fehlermeldung ausgegeben. Für die Ausgabe der Matrix wird eine for-Schleife über insgesamt elf Durchläufe genutzt, wobei neun für die Zeilen des Sudokus stehen und zwei für die Trennstriche zwischen den 3x3 Blöcken. Hierbei gibt es die Variable *verschiebung*, die dafür sorgt, dass trotz der Trennstriche die korrekten Zeilen der Matrix ausgegeben werden. Das erfolgt, indem die Variable jeweils bei dem Einfügen der horizontalen Trennstrichen hochgezählt wird. Dann wird jeweils die Matrix in der Zeile *i-verschiebung* ausgegeben. Die Trennstriche werden bei den Durchläufen 3 und 7 ausgegeben. Die vertikalen Abtrennungen der 3x3 Blöcke werden manuell durch senkrechte Striche in der Ausgabe eingefügt. Nach der Ausgabe wird die Datei wieder geschlossen.

9 Tests (Jari)

To do: Jari